

1.2 端到端评估

端到端评估将检索和生成模块作为一个整体来评估，以确定RAG系统在实际应用场景中的综合性能。

从内容生成目标的角度来看，评估可以分为无标签内容和有标签内容，未标记内容评估指标包括答案保真度、答案相关性、无害性等，而标记内容评估指标包括准确性和EM。

另外，从评估方法的角度来看，端到端评估可以分为手动评估和使用LLM的自动评估，而针对不同的领域，可以根据RAG在特定领域的应用，采用特定的评估指标，例如用于问答任务的EM，用于摘要的UniEval和E-F1任务，以及用于机器翻译的BLEU，这些指标有助于了解RAG在各种特定应用场景中的性能。

- 端到端评估通常采用用户满意度调查、任务完成率（Task Completion Rate）和对话质量评分（Dialogue Quality Score）等指标。
- 端到端评估强调实际应用效果，评估RAG系统在处理复杂查询或任务时的整体效能，包括响应的准确性、信息的完整性以及用户交互体验的流畅性。

2. 主流评估框架

工具推荐：

[ragas](#)

[deepeval](#)

[trulens](#)

2.1 RAGAS 框架

2.1.1 简介

RAGAS（**R**etrieval **A**ugmented **G**eneration **A**ssessment）是一个评估框架，专为检索增强生成RAG系统设计，用于无需参考答案的情况下评估这些系统的性能。它通过自动化的方式评估生成的文本在三个质量维度上的表现：忠实度（答案与上下文的一致性）、答案相关性（答案是否直接回答了问题）和上下文相关性（检索到的上下文是否专注且有用）。

2.1.2 评估策略

（1）三个质量维度

- **忠实度**：回答应基于给定上下文，避免幻觉并确保检索上下文可以作为生成答案的依据。
- **答案相关性**：生成的答案应该针对提供的实际问题。
- **上下文相关性**：检索到的上下文应该尽可能专注，包含尽可能少的无关信息。

(2) 自动化度量方法

使用LLM的提示来全自动化地测量这些质量方面，实验中使用OpenAI API提供的gpt-4-turbo-preview模型。

(3) 维度-1：忠实度

- 如果答案 $a_s(q)$ 中的声明可以从上下文 $c(q)$ 推断出来，则称答案对上下文是忠实的。
- 首先使用LLM提取一组声明 $S(a_s(q))$ ，目的是将较长的句子分解为更短、更聚焦的断言。
- 用于提取声明的提示：“给定一个问题和答案，从给定答案中的每个句子创建一个或多个声明。”
- 使用验证函数 $v(s_i, c(q))$ 确定 S 中的 s_i 是否可以从 $c(q)$ 推断出来。
- 忠实度得分 F 的计算方式是 $F = \frac{|V|}{|S|}$ ，其中 $|V|$ 是LLM支持的声明数量， $|S|$ 是声明的总数。

>>> 举例讲解 <<<

场景设定：

假设我们正在使用一个LLM来回答关于“北京故宫”的问题。

- 问题 q 是：故宫是在哪个朝代建成的？
- 系统生成的答案 $a_s(q)$ 是：故宫建成于明朝，由朱棣下令建造。

步骤 1：提取声明

使用LLM将答案中的每个句子分解成独立的、更精确的声明。我们可能会用下面的prompt去做这件事：

给定一个问题和答案，请从给定答案的每个句子中创建一个或多个声明：

问题：故宫是在哪个朝代建成的？

答案：故宫建成于明朝，由朱棣下令建造。

在这个例子中，可能的声明有：

1. 故宫建成于明朝。
2. 故宫是朱棣下令建造的。

步骤 2：验证声明

使用LLM对每个声明进行验证，看看这些声明是否可以从给定的上下文中推断出来。这里的上下文 $c(q)$ 可能是关于故宫历史的正式文档或可靠来源的历史记录。

prompt可能是：

请考虑给定的上下文和以下声明，然后确定它们是否得到上下文中信息的支持。在得出结论（是/否）之前，请为每个声明提供简短的解释。请按照指定格式为每个声明最终得出结论。不要偏离指定的格式。

声明：故宫建成于明朝。

声明：故宫是朱棣下令建造的。

...

步骤 3：计算忠实度得分

根据LLM的验证，计算得到支持的声明数 $|V|$ 和总声明数 $|S|$ 。在我们的例子中，如果历史记录证实了这两个声明，那么我们会两个支持的声明。

最后的忠实度得分 F 计算为 $F = \frac{|V|}{|S|}$ 。在这个例子中，如果两个声明都被验证为正确（即，历史记录确实表明故宫是明朝建成的，并且是由朱棣下令建造的），那么 $|V| = 2$ 并且 $|S| = 2$ ，所以忠实度得分 F 为 1 或者 100%。

这个忠实度得分表明生成的答案在历史上是准确的，完全基于正确的上下文信息。如果在验证过程中发现某些声明并不准确，或者上下文中没有足够的信息来支持这些声明，那么 $|V|$ 将会小于 $|S|$ ，得分会相应降低。

(4) 维度-2：答案相关性

- 答案 $a_s(q)$ 的相关性是指它是否直接以恰当的方式回答了问题 q 。
- 评估答案相关性不考虑事实性，但会惩罚答案不完整或包含冗余信息的情况。
- 为了估计答案相关性，基于答案 $a_s(q)$ ，通过LLM生成 n 个可能的问题 q_i ，使用以下提示："基于给定的答案生成问题：答案: [answer]"。
- 使用OpenAI API提供的text-embedding-ada-002模型获取所有问题的嵌入，计算 q_i 与原始问题 q 之间的相似性，作为余弦相似度。
- 答案相关性得分 AR 通过公式 $AR = \frac{1}{n} \sum_{i=1}^n sim(q, q_i)$ 计算，此指标评估生成答案与初始问题对齐的程度。

###

>>> 举例讲解 <<<

场景设定：

假设我们正在使用一个LLM来回答关于“太阳系”的问题。问题 q 是：“太阳系中最大的行星是哪一个？”系统生成的答案 $a_s(q)$ 是：“木星是太阳系中最大的行星，其体积和质量均是最大的。”

步骤 1：生成潜在问题

我们要求LLM基于答案 $a_s(q)$ 生成一系列潜在的问题 q_i 。比如，我们可以给LLM以下提示：

为给定答案生成3个问题：

答案：木星是太阳系中最大的行星，其体积和质量均是最大的。

LLM可能会生成如下潜在问题：

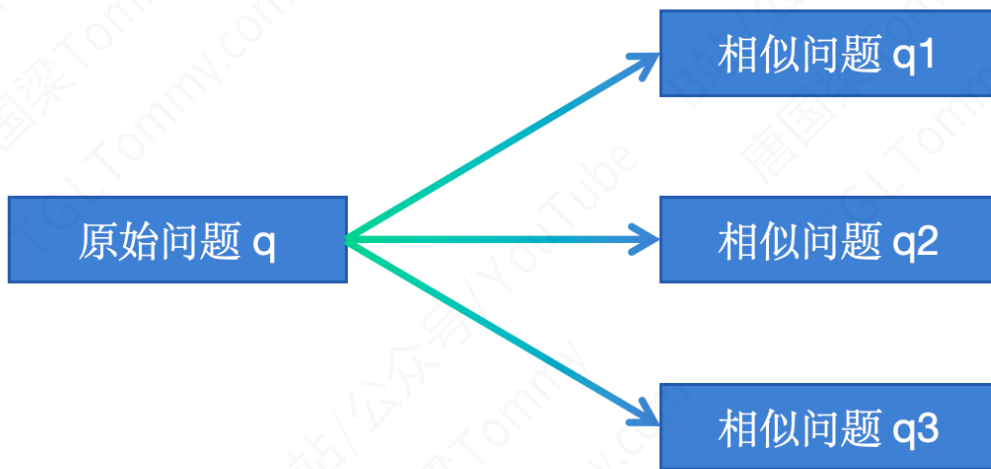
1. 太阳系中哪个行星最大？
2. 哪个行星在体积和质量上是太阳系中的领导者？
3. 太阳系中最重的行星是哪一个？

步骤 2：获取问题嵌入

使用OpenAI API提供的text-embedding-ada-002模型获取原始问题 q 以及每个潜在问题 q_i 的嵌入。

步骤 3：计算相似度

对于每个潜在问题 q_i ，计算其与原始问题 q 在嵌入空间中的余弦相似度 $\text{sim}(q, q_i)$ 。



步骤 4：计算答案相关性得分

答案相关性得分 AR 通过下面的公式计算：

$$AR = \frac{1}{n} \sum_{i=1}^n \text{sim}(q, q_i)$$

其中 n 是生成的潜在问题的数量。

例子中的计算：

假设我们计算出的相似度分别是：

1. $\text{sim}(q, q_1) = 0.95$
2. $\text{sim}(q, q_2) = 0.90$
3. $\text{sim}(q, q_3) = 0.85$

那么答案相关性得分 AR 为：

$$AR = \frac{1}{3} (0.95 + 0.90 + 0.85) = 0.90$$

这个得分反映了生成的答案与原始问题的对齐程度。如果 AR 得分接近1，这意味着生成的答案非常相关；如果得分低，可能意味着答案虽然正确，但可能不完全针对问题，或包含了一些与问题不直接相关的信息。在我们的例子中， AR 得分是0.90，这意味着生成的答案与原始问题非常相关。

(5) 维度-3：上下文相关性

- 上下文 $c(q)$ 的相关性是指它在多大程度上仅包含回答问题 q 所需的信息。
- 评估上下文相关性时，LLM会从上下文 $c(q)$ 中提取一组句子 S_{ext} ，这些句子对回答问题 q 至关重要。
- 使用的Prompt："请从提供的上下文中提取可能有助于回答以下问题的相关句子。如果没有找到相关句子，或者你认为这个问题无法从给定的上下文中得到回答，请返回短语“信息不足”。在提取候选句子时，不允许对给定上下文中的句子进行任何更改。"
- 上下文相关性得分 CR 通过公式 $CR = \frac{\text{number of extracted sentences}}{\text{total number of sentences in } c(q)}$ 计算，此指标旨在惩罚包含的冗余信息。

>>> 举例讲解 <<<

场景设定：

假设我们正在使用一个语言模型来回答关于“长城”的问题。问题 q 是：“长城是在哪个朝代修建的？”系统检索到的上下文 $c(q)$ 包含了关于长城的一段历史描述，其中包含多个句子，有的与长城的建设时间相关，有的描述了长城的其他特征。

步骤 1：提取关键句子

我们需要从提供的上下文 $c(q)$ 中提取出对回答问题 q 至关重要的句子 S_{ext} 。我们可以给LLM以下提示：

请从提供的上下文中提取相关句子，这些句子可能有助于回答以下问题。如果没有找到相关句子，或者您认为问题不能用给定上下文回答，请返回短语“信息不足”。在提取候选句子时，不允许对给定上下文中的句子进行任何修改。

步骤 2：LLM执行句子提取

LLM根据提示从上下文中提取相关句子。例如，如果上下文中有10个句子，LLM可能确定其中3个句子直接回答了问题。

步骤 3：计算上下文相关性得分

上下文相关性得分 CR 是提取的相关句子数量与上下文中句子总数的比例。公式如下：

$$CR = \frac{\text{提取的相关句子数量}}{\text{上下文中句子总数}}$$

假设上下文 $c(q)$ 包含以下句子：

- 长城是中国的象征。
- 它的建设始于春秋时期。
- 秦始皇统一六国后开始修建长城。
- 长城在明朝得到了大规模的修缮和扩建。

5. 长城是世界上最长的墙。
6. 它是一项巨大的防御工程。
7. ...

如果问题是关于长城修建的朝代，那么句子2和句子4是直接相关的。如果LLM正确提取了这两个句子作为回答问题所需的信息，而总共有10个句子，那么：

$$CR = \frac{2}{10} = 0.2$$

这个得分表明，从上下文中提取的相关信息在总信息中所占的比例。得分越高，表示提取的上下文越专注于问题所需的信息。在我们的例子中，上下文相关性得分是0.2，意味着从整个上下文中提取的信息对问题的相关性只有20%。这可能表明上下文中包含了很多其他不相关的信息，或者LLM在提取相关信息时过于保守。

2.1.3 ragas的特点

(1) 优势

- 提高了评估效率：RAGAS可以快速评估RAG系统，不需要等待人工注释。
- 节省成本：自动化的评估过程比人工注释便宜得多。
- 高度一致性：实验表明，RAGAS的评估结果与人类评估者的判断在大多数情况下非常一致。

(2) 劣势

- 在评估上下文相关性时可能存在挑战：在某些情况下，特别是在上下文较长时，ChatGPT在选择关键句子方面可能会遇到困难。
- 对于微妙的差异可能不够敏感：在比较答案相关性时，如果两个答案候选之间的差异非常细微，可能会降低评估的一致性。
- 可能存在随机性：如果LLM给出相同分数的答案候选，需要随机打破平局，这可能引入一些随机性，从而影响评估的确定性。

七、项目实战

1. FAQ-ChatBot